

SUPPLEMENTARY · MODULE 6

Resilience and Performance

Keep services up when dependencies fail: timeouts, retries, circuit breakers, bulkheads, caching, and the SLOs a bank will actually hold you to.



Day 3 Objectives and Capstone Progress

DAY 3 OBJECTIVES



1 Risk Service Development

Build the Risk Service with rules engine and scoring logic.



2 Advanced Rules & Scoring

Implement configurable risk rules and scoring algorithms.



3 Event-Driven Processing

Consume transaction events and publish risk evaluation results.



4 Notification Service

Build Notification Service to send alerts based on risk results.



5 End-to-End Saga Flow

Complete the saga flow: Transaction → Risk Evaluation → Notification.



6 Testing & Observability

Add tests, verify integrations, and monitor services with Actuator.

CAPSTONE PROGRESS

Overall Progress

60%



What We Will Achieve by End of Day 3



Risk evaluation engine working



Rules configuration and scoring



Events consumed and published



Alerts generated for high-risk transactions



End of Day 3 Goal

Complete Risk Service and integrate with Transaction Service. Have risk evaluation results published and ready for notifications.

DESIGNING FOR FAILURE

Services may be unavailable, slow, or overloaded -- resilience means defining what happens next, not hoping it never occurs.



ASSUME FAILURE IS NORMAL

- Services may be unavailable, responses may be slow, databases may reject connections
- Message brokers may back up, external APIs may throttle, infrastructure may restart



RESILIENCE BUILDING BLOCKS

- Timeouts, retries, circuit breakers, bulkheads, rate limits
- Health checks, metrics, logging, distributed tracing, alerting

KEY TAKEAWAY Services may be unavailable, slow, or overloaded -- resilience means defining what happens next, not hoping it never occurs.

Designing for Failure in Financial Workflows

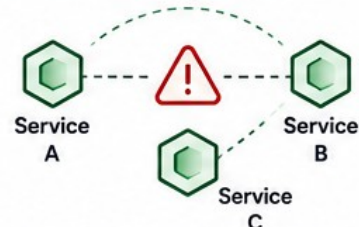
1. Expect Failure



Failures are normal in distributed systems.

- Network issues
- Service outages
- Slow responses
- Data inconsistencies

2. Contain the Impact



Isolate failures to prevent cascading across services.

- Fault isolation
- Bulkheads and timeouts
- Circuit breakers

3. Detect Quickly



Identify issues early to minimize customer impact.

- Health checks
- Logging and metrics
- Alerts and dashboards

4. Recover Gracefully



Recover automatically and safely whenever possible.

- Retries with backoff
- Idempotent operations
- Compensating actions

Failure Handling Strategies



Prevent

Validate inputs, enforce rules, and reduce the chance of failure.



Retry

Retry transient failures with exponential backoff.



Degrade

Provide limited functionality when dependencies fail.



Compensate

Undo or adjust actions to maintain business consistency.



Notify

Inform users or systems to enable timely response.

Key Design Principles



Resilience by Design

Build systems that continue to operate under failure.



Idempotency

Ensure repeated requests have the same effect.



Data Consistency

Use appropriate consistency models and safeguards.



Observability

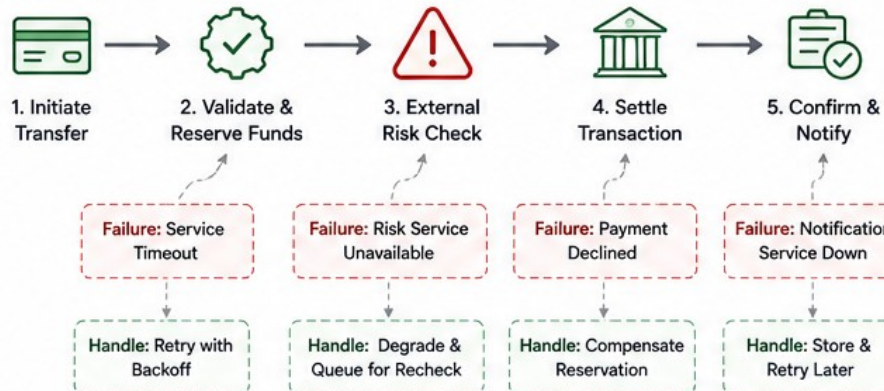
Measure, monitor, and understand system behavior.



User Communication

Set clear expectations and provide timely updates.

Financial Workflow Example



Outcomes



Maintain Trust

Reliable experiences build customer confidence.



Protect Money

Prevent loss, duplication, and fraud.



Ensure Continuity

Keep critical services available.



Enable Compliance

Meet regulatory and audit requirements.

FAILURE MODES IN DISTRIBUTED SYSTEMS

Immediate errors, slow responses, timeouts, and partial failures each demand a different, deliberately designed response.



COMMON FAILURE TYPES

- Immediate error response, slow response, timeout, partial failure
- Resource exhaustion, message processing failure, data inconsistency, configuration error

KEY TAKEAWAY Immediate errors, slow responses, timeouts, and partial failures each demand a different, deliberately designed response.

Distributed-System Failure Modes

1 Network Failures



What It Is

Issues that prevent services from communicating.

Typical Symptoms

- Timeouts
- Connection errors
- Partial partitions
- High latency

2 Service Failures



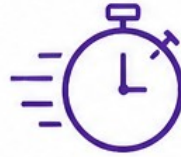
What It Is

A service crashes or becomes unavailable.

Typical Symptoms

- Health check failures
- 5xx errors
- Unreachable endpoints
- Restarts / crash loops

3 Timeouts



What It Is

A service takes too long to respond.

Typical Symptoms

- Request timeouts
- Thread pool exhaustion
- Cascading slowdowns
- Increased latency

4 Data Inconsistencies



What It Is

Data differs across services or is out of sync.

Typical Symptoms

- Stale reads
- Conflicting updates
- Missing or duplicate data
- Integrity constraint errors

5 Resource Exhaustion



What It Is

A service runs out of CPU, memory, threads, or connections.

Typical Symptoms

- High CPU / memory usage
- OutOfMemory errors
- Connection pool exhaustion
- Thread starvation

6 Overload



What It Is

Incoming load exceeds the system's capacity.

Typical Symptoms

- Slow responses
- Queue buildup
- 429 / 503 errors
- Shed load / backpressure

7 Partial Failures



What It Is

Some parts of the system work while others fail.

Typical Symptoms

- Intermittent errors
- Inconsistent behavior
- Hard to reproduce issues
- Split-brain effects

8 Message / Broker Failures



What It Is

The messaging system is slow, unavailable, or loses messages.

Typical Symptoms

- Publish/consume failures
- Message backlog
- Duplicate or missing messages
- Dead letters

9 Configuration Errors



What It Is

Incorrect or inconsistent configuration across services.

Typical Symptoms

- Wrong endpoints / ports
- Auth / permission failures
- Feature flags misconfigured
- Environment drift

10 Dependency Failures



What It Is

External dependencies are slow or unavailable.

Typical Symptoms

- Third-party timeouts
- Circuit breaker open
- API rate limit errors
- DNS / resolver failures

Common Characteristics



Unpredictable



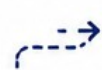
Can happen
any time



Affect multiple
services



Hard to
reproduce



Require end-to-end
visibility

Design for Failure



Fail fast



Retry with
backoff



Circuit
breakers



Bulkheads &
limits



Observability &
alerts



Graceful
degradation

NETWORK LATENCY AND UNRELIABLE COMMUNICATION · CASCADING FAILURE



WHY NETWORK CALLS ARE DIFFERENT

- Latency, jitter, and packet loss are inherent to any network hop
- **A remote call can fail in ways a local call simply cannot:** DNS failure, connection reset, partial response



HOW CASCADING FAILURE SPREADS

- A slow or failing dependency ties up threads, connections, and memory in the caller
- The caller becomes unhealthy too, and the failure propagates to ITS callers
- Timeouts, circuit breakers, and bulkheads exist specifically to stop this chain

KEY TAKEAWAY In a monolith, calls happen in memory; in microservices, every call crosses a network that can delay, drop, or corrupt it. • A slow Fraud Detection Service can exhaust Transaction Service's threads, which then can't serve any request -- healthy or not.

TIMEOUT PATTERN

Without a timeout, slow dependencies can silently consume threads, memory, and connection pools until the caller itself becomes unhealthy.



WHERE TIMEOUTS APPLY

- HTTP calls, database queries, message broker operations, external API calls
- File operations and cache operations also need timeouts



BANKING EXAMPLE

- **Transaction Service -> Account Service:** waits up to 2 seconds
- If no response, timeout handling takes over -- Transaction Service does not wait indefinitely

KEY TAKEAWAY Without a timeout, slow dependencies can silently consume threads, memory, and connection pools until the caller itself becomes unhealthy.

Timeout Strategies

1. Timeout Types



Connection Timeout

Maximum time to establish a connection.



Read Timeout

Maximum time to wait for a response after request is sent.



Overall Timeout

Total time allowed for the entire operation.

2. Recommended Approach



Set Timeouts Appropriately

Based on service behavior and SLA requirements.



Differentiate by Operation

Use different timeouts for read, write, and health checks.



Fail Fast

Use reasonable timeouts to free resources quickly.



Retry or Fallback

Combine timeouts with retry or fallback strategies.



Observe & Tune

Monitor latency and adjust timeouts over time.

3. Timeout Configuration Guidelines

Timeout Type	Typical Range	Guideline
 Connection Timeout	1 – 3 seconds	Keep low to quickly detect network or service issues.
 Read Timeout	2 – 10 seconds	Based on expected processing time and percentiles (P95/P99).
 Overall Timeout	= Connection + Read + Buffer	Include buffer for retries and downstream calls.

Best Practices

- ✓ Set timeouts shorter than user or upstream timeouts.
- ✓ Use different timeouts for internal vs external calls.
- ✓ Avoid very long timeouts that tie up resources.
- ✓ Document timeout values and review regularly.

4. Timeout + Retry + Backoff



Request with Timeout



Timeout Occurs



Retry with Backoff



Success (or Fallback)



Use exponential backoff with jitter to reduce load and avoid retry storms.

5. Example Configuration (Spring Boot)

```
# Example for RestTemplate / WebClient
connect-timeout: 2000 # 2 seconds
read-timeout: 5000 # 5 seconds

# Example for WebClient
response-timeout: 5s # 5 seconds

# Resilience4j Timeout
resilience4j.timeout.instances.myService.
  timeout-duration: 3s
  cancel-running-future: true
```

6. Outcome



Prevents cascading failures



Improves system responsiveness



Frees up resources quickly



Better user experience



Key Takeaway

Use appropriate timeouts, combine with retries and fallbacks, and continuously observe and tune to build resilient and responsive distributed systems.

CHOOSING EFFECTIVE TIMEOUT VALUES · RETRY PATTERN · EXPONENTIAL BACKOFF AND JITTER



TIMEOUT BUDGET EXAMPLE

- **Total budget:** 3000 ms
- **Customer lookup:** 500 ms, Account validation: 700 ms, Fraud check: 800 ms
- Remaining budget covers response processing



SELECTION FACTORS

- Business requirements, expected latency, user experience, downstream behavior, SLA targets
- Short timeouts fail fast but risk false rejects; long timeouts tie up resources



RETRYABLE FAILURES

- Temporary network error, HTTP 503, HTTP 429 with retry-after guidance
- Short database connection interruption, temporary broker disconnect



NON-RETRYABLE FAILURES

- Validation errors, authentication failure, authorization failure
- Resource not found, business rule rejection



BACKOFF PROGRESSION EXAMPLE

- Attempt 1 fails -> wait 200 ms
- Attempt 2 fails -> wait 500 ms
- Attempt 3 fails -> stop and report failure



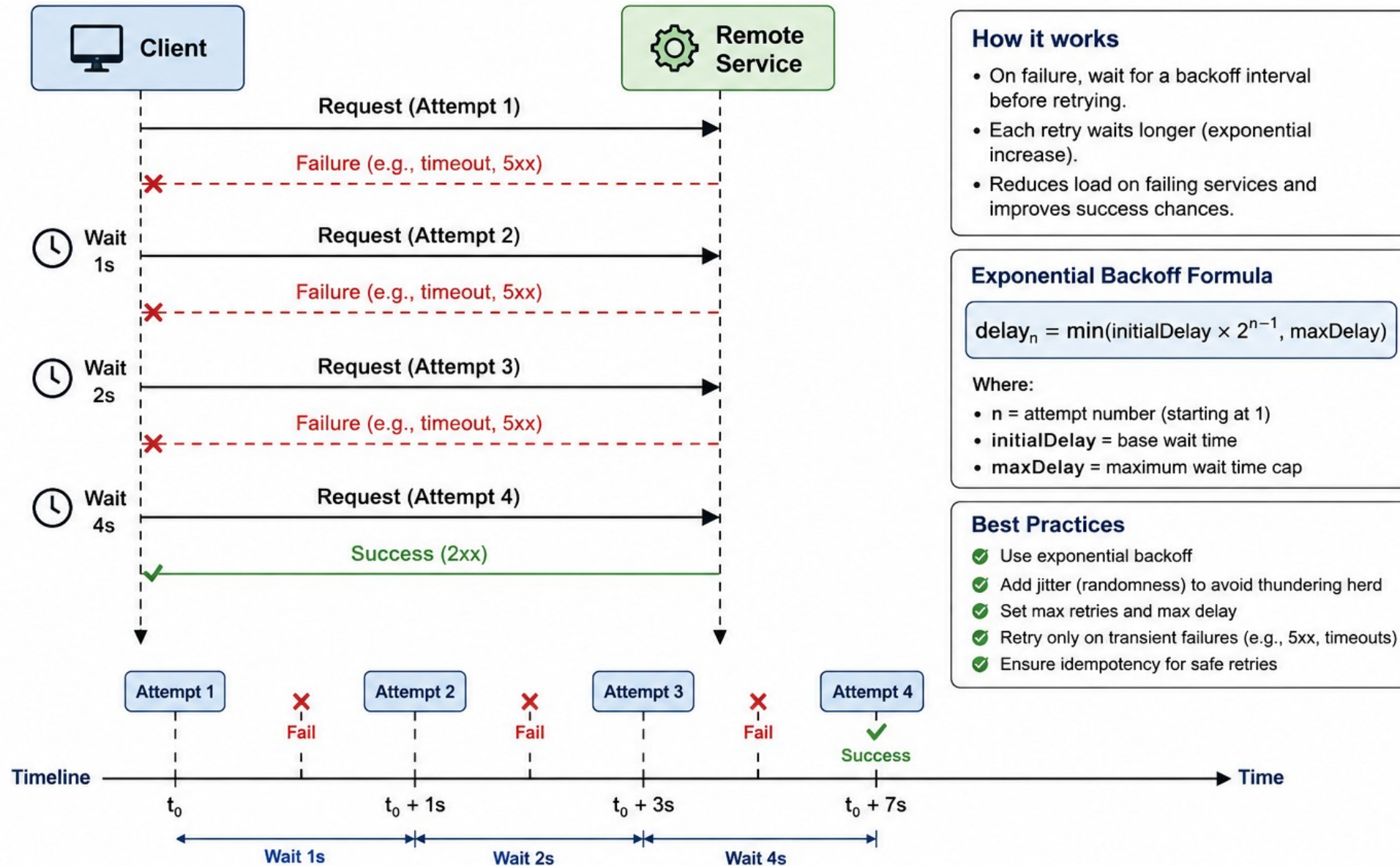
WHY JITTER MATTERS

- Adds randomness to retry delays so many instances don't retry in lockstep
- Prevents a coordinated retry spike from overwhelming a recovering dependency

KEY TAKEAWAY If the customer-facing API must respond in 3 seconds, downstream timeouts must be carved out of that same 3-second budget.

- Retrying a temporary email provider outage is safe; retrying a money transfer without idempotency can create duplicate transactions.
- Without jitter, many service instances retrying on the exact same schedule can create a synchronized retry storm that re-overloads the dependency.

Retry Mechanisms and Exponential Backoff (awareness)



WHEN RETRIES MAKE FAILURES WORSE · CIRCUIT BREAKER PATTERN



HOW RETRIES BACKFIRE

- Retrying every exception, including non-transient ones
- No maximum retry limit, or retrying non-idempotent operations
- Combining long timeouts with multiple retries multiplies total wait time



WHY CIRCUIT BREAKERS HELP

- Prevent cascading failures, thread exhaustion, and excessive retries
- Stop slow request buildup and avoid overloading an already-failing service

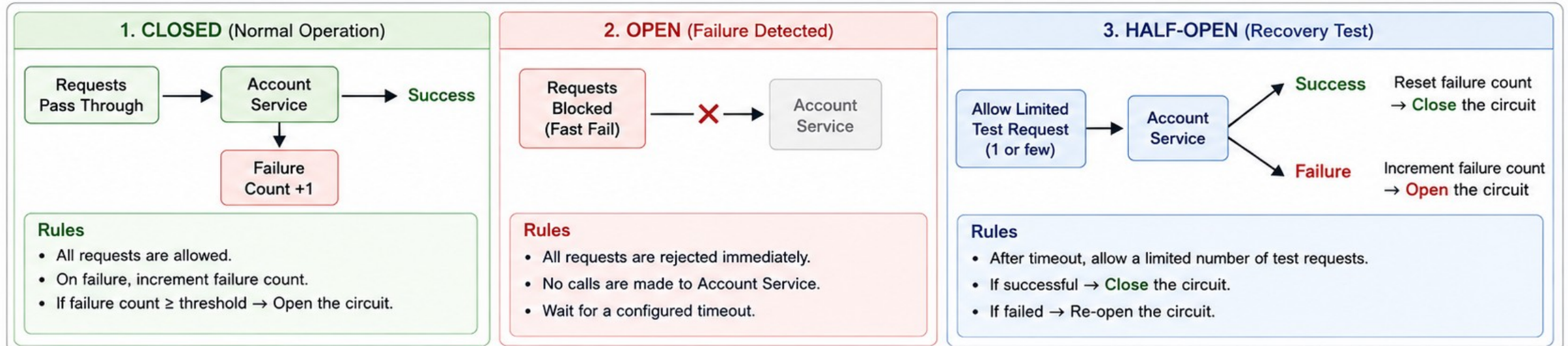
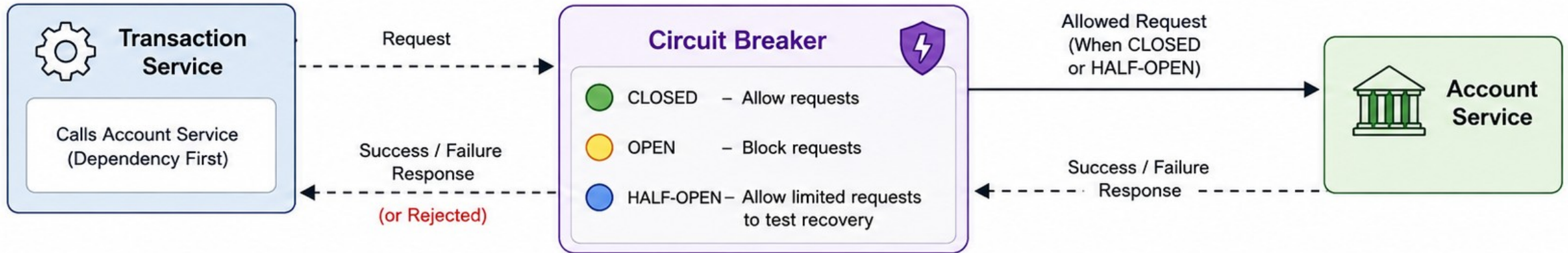


BANKING EXAMPLE

- If Fraud Detection Service fails repeatedly, the circuit opens
- Transactions may be rejected, marked pending review, or handled under a fallback policy

KEY TAKEAWAY Retrying every exception, with no limit, from many instances at once, amplifies load on a dependency that's already struggling to keep up. • When failures exceed a threshold, the circuit opens and blocks calls immediately -- protecting both the caller's resources and the struggling dependency.

Circuit Breaker Pattern (Account Service dependency first)



CIRCUIT BREAKER STATES AND TRANSITIONS · BULKHEAD PATTERN



THE THREE STATES

- **Closed:** all calls allowed, success/failure rates monitored
- **Open:** calls blocked immediately, fail fast or use a fallback
- **Half-open:** a small number of test calls allowed after a wait duration



RESILIENCE4J CONFIGURATION CONCEPT

- **failure-rate-threshold:** 50, sliding-window-size: 20, wait-duration-in-open-state: 30s

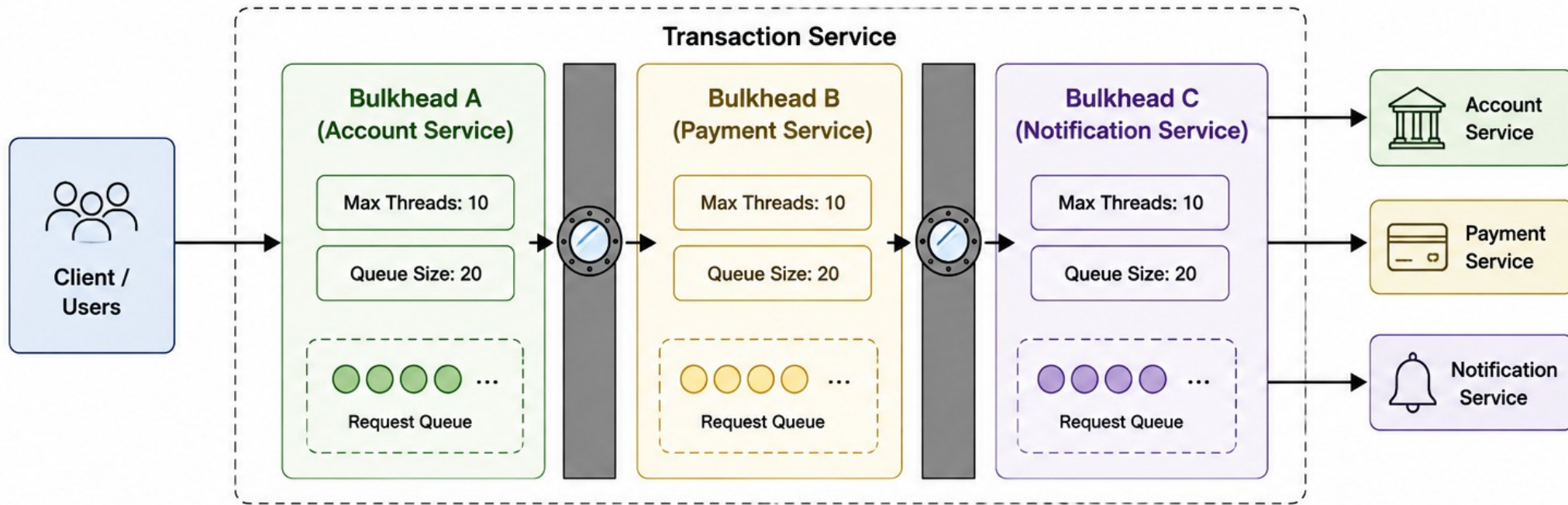


WHY BULKHEADS MATTER

- Without isolation, one slow dependency can consume all available threads
- The term comes from ship compartments; in software it means separate thread pools, connection pools, or concurrency limits

KEY TAKEAWAY Half-open allows a small number of test calls through -- success closes the circuit, failure reopens it immediately. • If reporting export hangs, Account Service should still serve balance lookups -- bulkheads are what make that guarantee possible.

Bulkhead Pattern (conceptual)



How It Works

1

Each downstream dependency is isolated behind its own bulkhead with separate resource limits.

2

If one dependency is slow or fails, its bulkhead exhausts resources independently.

3

Other bulkheads still have available resources and continue to serve requests.

4

Failures are contained to the failing bulkhead; system remains resilient.

Key Purpose



Prevent a failure or high load in one external dependency from exhausting resources and impacting others.

Common Resource Limits



• Max Threads / Connections



• Queue Size



• Timeout per Call



• Rate Limit (optional)



Result: Improved isolation, better fault tolerance, and predictable performance.

SEMAPHORE VS. THREAD-POOL BULKHEADS · RATE LIMITING AND THROTTLING · FALLBACK STRATEGIES



TWO BULKHEAD IMPLEMENTATIONS

- **Semaphore bulkhead:** limits concurrent access without a separate thread pool, lightweight
- **Thread pool bulkhead:** uses a separate thread pool, can queue calls before rejection



WHEN TO USE EACH

- **Semaphore:** reactive or non-blocking applications
- **Thread pool:** blocking operations that need full isolation of thread resources



COMMON ALGORITHMS

- Fixed window, sliding window, token bucket, leaky bucket
- **Token bucket:** tokens refill over time; each request consumes one, allowing controlled bursts



WHERE IT'S APPLIED

- API gateway, load balancer, service mesh, application code, or the external API client
- Exceeding the limit returns 429 Too Many Requests



FALLBACK OPTIONS

- Return cached reference data, mark a transaction as pending, return a controlled error
- Queue work for later processing, or disable a non-critical feature

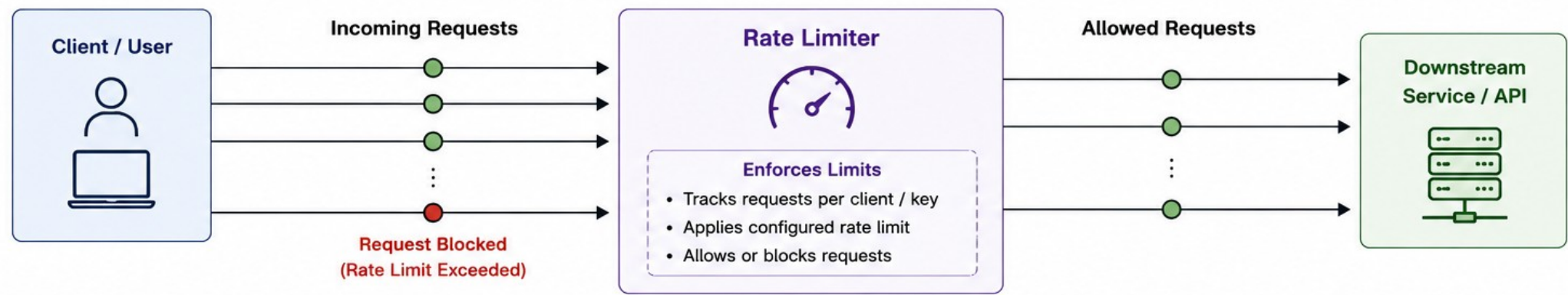


THE GOLDEN RULE

- Fallbacks must be business-approved and must never invent or fabricate critical data

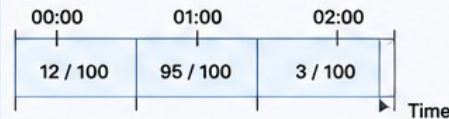
KEY TAKEAWAY A semaphore is lightweight and non-blocking by default; a thread pool bulkhead adds true isolation at the cost of managing its own threads. • Fixed window, sliding window, token bucket, and leaky bucket algorithms each shape traffic differently, but all exist to protect capacity. • Returning a fabricated account balance is never acceptable; hiding a marketing offer when the recommendation service is down usually is.

Rate Limiting Overview (conceptual)



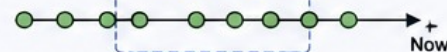
Common Rate Limiting Algorithms

1. Fixed Window Counter



Counts requests in fixed time windows. Simple but can allow bursts at window boundaries.

2. Sliding Window Log



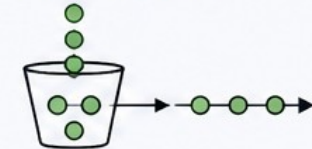
Stores timestamps of requests in the current window. More accurate but uses more memory.

3. Sliding Window Counter



Weighted count of current and previous windows. Balances accuracy and efficiency.

4. Token Bucket



Tokens added at a steady rate. Requests consume tokens. Allows bursts up to bucket capacity.

Key Benefits

- ✓ Prevents abuse and protects system resources
- ✓ Ensures fair usage among clients
- ✓ Enhances stability and performance
- ✓ Supports multi-tenant and API monetization

Common Use Cases

- API protection
- Per-user / per-tenant limits
- Prevent brute force / abuse
- Control traffic to downstream services



Typical Configuration

Scope / Key	e.g., IP address, User ID, API Key
Limit	e.g., 100 requests
Time Window	e.g., per minute
Action on Limit Exceeded	e.g., Reject (429), Throttle, Queue
Headers	e.g., Retry-After, X-RateLimit-*

Rate Limit Response Example

HTTP Status: 429 Too Many Requests

Headers: Retry-After: 60
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1715698800

Body (example): {
 "error": "rate_limit_exceeded",
 "message": "Too many requests. Please try again later."
}

GRACEFUL DEGRADATION · LOAD SHEDDING · RESILIENCE4J OVERVIEW



WHAT GRACEFUL DEGRADATION MEANS

- The system provides reduced functionality instead of failing completely
- For critical banking operations, degradation must still follow business and compliance rules



HOW LOAD SHEDDING WORKS

- The service monitors its own saturation (CPU, threads, queue depth)
- When capacity is exceeded, lower-priority requests are rejected early, before they consume resources
- Protects the service's ability to serve its most important traffic



WHY RESILIENCE4J

- Lightweight and fast, fault tolerant by design
- Rich metrics and events, production ready
- Modules for Retry, CircuitBreaker, Bulkhead, RateLimiter, and TimeLimiter

KEY TAKEAWAY If Recommendation Service is unavailable, the dashboard still shows balances and transactions -- it simply hides personalized offers.

- When demand exceeds capacity, shedding low-priority traffic protects the requests that matter most from a total system collapse.
- Timeout, retry, circuit breaker, bulkhead, and rate limiter modules share one consistent configuration style across a Spring Boot service.

Safe Fallback Rules - never invent a successful financial outcome



Core Principle

When a dependency is unavailable or uncertain,
do not assume success.
Return a safe failure or pending state.

Mindset

- Don't guess.
- Don't fabricate.
- Don't create money, move money, or change balances.



Golden Rule

Fail safe.
Be conservative.
Protect financial integrity.

Scenario (Dependency Fails)	Never Do This	Do This Instead (Safe Fallback)	Client Response
 Account Service unavailable (balance or account validation)	 Assume account exists and has sufficient funds. Do not proceed.	 Return 503/Service Unavailable or 424/Failed Dependency. Do not create or confirm transaction.	 Show failure to validate. Ask user to retry. No money moved.
 Payment Gateway unavailable (charge authorization)	 Assume payment authorized. Do not capture/settle.	 Do not capture. Return 503/Service Unavailable. Keep transaction in PENDING state for retry or user action.	 Show payment pending or failed to process. No charge applied.
 Fraud/Risk Service unavailable (risk evaluation)	 Assume low risk. Do not approve based on guess.	 Return 503/Service Unavailable or flag as PENDING_REVIEW. Do not auto-approve.	 Show pending review or try again later. No approval granted.
 Ledger Service unavailable (posting transaction)	 Assume transaction posted. Do not record success.	 Do not mark as posted. Store in PENDING state. Retry posting; ensure idempotency.	 Show transaction pending. No success confirmation.
 Notification Service unavailable (send receipt/alert)	 Assume notification delivered. Do not confirm sending.	 Accept the request. Store for retry (outbox). Do not mark as sent.	 Show notification pending. It will be sent later. No success confirmation.

Safe Fallback Checklist



- ✓ When in doubt, stop and fail safe.
- ✓ Return failure/pending, never success.
- ✓ Do not change balances or create commitments.
- ✓ Be explicit, transparent, and retryable.



Remember

A wrong success is worse than a temporary failure.
Protect trust. Protect money.

IMPLEMENTING TIMEOUTS WITH RESILIENCE4J · IMPLEMENTING RETRIES WITH RESILIENCE4J · IMPLEMENTING CIRCUIT BREAKERS



CONFIGURATION CONCEPT

- `resilience4j.timelimiter.instances.accountService.timeout-duration: 2s`
- `@TimeLimiter(name = "accountService")` on the client method
- Pairs naturally with a fallback method for the timeout case



SPRING BOOT EXAMPLE

- `@CircuitBreaker(name = "externalService", fallbackMethod = "fallback")`
- `sliding-window-size`, `minimum-number-of-calls`, `failure-rate-threshold`, `wait-duration-in-open-state`



CONFIGURATION CONCEPT

- **`resilience4j.retry.instances.accountService.max-attempts: 3`**
- **`resilience4j.retry.instances.accountService.wait-duration: 500ms`**
- Combine with exponential backoff and jitter, and target only transient failures



METRICS AND EVENTS

- Micrometer auto-registers `resilience4j.circuitbreaker.calls`, `.state`, `.successful.calls`, and more
- **Key events:** `OnSuccess`, `OnError`, `OnSlowCall`, `OnCallNotPermitted`, `OnStateTransition`

KEY TAKEAWAY `@TimeLimiter` combined with a fallback method turns a hung downstream call into a controlled, predictable failure instead of an indefinite wait. • `max-attempts` and `wait-duration` in `application.yml` are all it takes to add safe, bounded retry behavior to a Spring Boot client call. • A sliding window of recent calls, a failure-rate threshold, and a `wait-duration-in-open-state` fully define a Resilience4j circuit breaker.

IMPLEMENTING BULKHEADS · IMPLEMENTING RATE LIMITERS · COMBINING RESILIENCE PATTERNS SAFELY



CONFIGURATION EXAMPLE

- `resilience4j.bulkhead.instances.externalService.max-concurrent-calls: 10`
- **type:** SEMAPHORE or THREADPOOL, with queue-capacity for thread-pool bulkheads
- Use BulkheadFullException to provide a graceful fallback when capacity is exhausted



RECOMMENDED EXECUTION ORDER

1. Timeout -- fail fast
2. Circuit Breaker -- stop failing fast
3. Retry -- recover from transient failures
4. Bulkhead -- isolate and contain
5. Rate Limiter -- control traffic
6. Fallback -- provide graceful behavior



CONFIGURATION EXAMPLE

- `resilience4j.ratelimiter.instances.externalService.limit-for-period: 10`
- **limit-refresh-period:** 1s, timeout-duration: 500ms, timeout-behavior: WAIT or FAIL_FAST



COMMON ANTI-PATTERNS

- Retrying non-idempotent operations, retries without backoff or jitter
- Unlimited bulkhead thread pools, overusing too many patterns at once

KEY TAKEAWAY `max-concurrent-calls` and `max-wait-duration` cap how many requests can be in flight to one dependency at any moment. `limit-for-period` and `limit-refresh-period` define a token bucket; excess calls wait, or fail fast with `RequestNotPermittedException`. Applying patterns in this recommended sequence protects the provider first, protects your own threads second, and degrades gracefully last.

RESILIENCE CONFIGURATION AND TUNING · TESTING FAILURE AND RECOVERY · CHAOS ENGINEERING FUNDAMENTALS



TUNING WITH DATA

- **Collect:** metrics, logs, and traces (p95/p99 latency, errors, timeouts)
- Analyze latency distributions and failure types; adjust one parameter at a time
- Validate improvements without introducing new issues, then document the decision



THE CHAOS ENGINEERING LIFECYCLE

1. Define hypotheses about expected system behavior
2. Inject controlled, safe failures (latency, pod kill, etc.)
3. Observe system response and identify weaknesses
4. Learn and improve; repeat continuously



TYPES OF FAILURES TO TEST

- Network failures, dependency failures, resource exhaustion
- Messaging failures, infrastructure failures (pod/node crashes)



COMMON EXPERIMENTS AND TOOLS

- Service kill, network latency, network partition, resource depletion, dependency failure
- **Tools:** Chaos Mesh, Litmus, Gremlin, Pumba, Toxiproxy



WHAT TO VALIDATE

- Do timeouts, retries, circuit breakers, bulkheads, and rate limiters behave as configured?
- Does the system recover automatically and quickly once the failure is removed?

KEY TAKEAWAY Measure, analyze, adjust one parameter at a time, validate, and document -- resilience tuning is a continuous feedback loop, not a one-time setup. • Plan, inject failure, observe, validate, recover -- a five-step approach turns 'we think it's resilient' into 'we know it's resilient.' • Define hypotheses, inject controlled failures, observe behavior, and learn -- turning unknown risks into known, fixable weaknesses.

Resilience Anti-Patterns

Resilience is not just about adding patterns—it's about using them wisely.



Anti-patterns create a false sense of resilience and increase risk.



They **hide problems**, waste resources, and hurt user experience.



Detect early in design, code reviews, testing, and operations.



Prefer **simple, observable**, failure-aware designs that fail safely.



Build a culture that values learning from failure, not hiding it.

COMMON RESILIENCE ANTI-PATTERNS

ANTI-PATTERN	WHAT IT LOOKS LIKE	WHY IT'S A PROBLEM	EXAMPLE	BETTER APPROACH
 Infinite Retries (No Backoff)	Retrying a failed call endlessly and immediately.	- Overloads failing service - Wastes resources - Delays recovery	<pre>while(true) { callService(); }</pre>	Use limited retries with exponential backoff and jitter. Fail fast when needed.
 Circuit Breaker Always Open	Configuring breakers too aggressively so they stay open.	- Services become unavailable - Prevents successful calls - Hides real system health	<pre>failureRateThreshold: 1 waitDurationInOpenState: 60s</pre>	Tune thresholds based on real latency/error distribution and allow traffic to probe recovery.
 Universal Fallback That Lies	Returning default or cached data for every failure.	- Hides issues from users - Returns stale or incorrect data - Breaks trust	<pre>catch(Exception e) { return defaultValue; }</pre>	Use meaningful fallbacks. Propagate errors when no safe alternative exists.
 Service Chaining (Deep Call Chains)	Synchronous calls across many services in a chain.	- High latency amplification - Cascading failures - Hard to debug	<pre>A → B → C → D → E</pre>	Use asynchronous messaging or API composition to reduce chaining and fan-out.
 No Timeouts Everywhere	Missing or excessive timeouts on external calls.	- Threads/resources get blocked - Slow failure detection - Cascades and timeouts	<pre>restTemplate.getForObject(...) // no timeout set</pre>	Set sensible connect, read, and write timeouts based on SLOs and dependencies.
 Oversized Bulkheads (No Isolation)	Large thread pools or semaphores shared across dependencies.	- One slow dependency starves others - No isolation - Throughput collapse	<pre>ThreadPool size = 500 (shared for all)</pre>	Use fine-grained bulkheads per dependency with correct limits and queues.
 No Observability (Blind Resilience)	No metrics, traces, or alerts for resilience components.	- Can't see failures or bottlenecks - Can't tune or improve - Surprises in production	<pre>Circuit breaker metrics = disabled</pre>	Instrument everything. Monitor, alert, and review regularly.
 Ignoring Dependency Characteristics	Using the same settings for all dependencies.	- One-size-fits-all doesn't work - Some dependencies need stricter or looser settings	<pre>timeout = 2s retries = 3 (all services)</pre>	Understand each dependency. Customize timeouts, retries, and limits accordingly.

THE IMPACT

- ✗ Increased downtime and slow recovery
- ✗ Poor user experience
- ✗ Wasted compute and network resources
- ✗ Cascading failures
- ✗ Loss of trust and revenue



HOW TO AVOID THEM

- ✓ Understand your system and dependencies.
- ✓ Use resilience patterns deliberately, not by default.
- ✓ Start simple, measure, and tune.
- ✓ Test under failure (including chaos experiments).
- ✓ Continuously observe and learn.



REMEMBER

Resilience is about graceful failure and fast recovery—not about hiding problems.
Failing silently is not being resilient.

RESILIENCE BEST PRACTICES (DO INSTEAD)



Define SLOs and error budgets.



Tune based on real data.



Observe continuously and alert usefully.



Test failure scenarios regularly.



Design for simplicity and graceful degradation.



Share learnings and improve over time.



Build resilience.
Build confidence.
Build systems people trust.

MICROSERVICES PERFORMANCE FUNDAMENTALS · LATENCY, THROUGHPUT, AND RESOURCE UTILIZATION · PERFORMANCE BUDGETS



COMMON PERFORMANCE BOTTLENECKS

- Network calls (latency, chattiness), database queries (N+1, missing indexes)
- Thread pool sizing, caching (low hit rate, stampede), serialization overhead



BEST PRACTICES

- Design for loose coupling, use asynchronous messaging where possible
- Cache close to the data, right-size pools, paginate, set timeouts, profile hot paths



THE THREE DIMENSIONS

- **Latency:** how fast a request is served (p50, p95, p99)
- **Throughput:** how much work is done per unit time (requests per second)
- **Resource utilization:** how efficiently CPU, memory, disk, and network are used



TRADE-OFFS TO UNDERSTAND

- Lower latency may reduce throughput; higher throughput may consume more resources
- Keep headroom -- never run at 100% utilization



PERFORMANCE BUDGET FORMULA

- Performance Budget = Target (user expectation) - Safety Margin - Variability Buffer
- **Types:** latency budget, throughput budget, availability budget, error budget, resource budget

KEY TAKEAWAY Performance is a feature -- optimizing end-to-end user experience matters more than tuning any single service in isolation. • Lowering latency often needs more resources; pushing throughput higher needs efficient resource usage -- optimizing one affects the others. • If the end-to-end target is 500 ms, each service in the chain gets its own slice -- and the sum must never exceed the total budget.

CONNECTION POOLING AND THREAD MANAGEMENT · JVM PERFORMANCE CONSIDERATIONS · HORIZONTAL VS. VERTICAL SCALING



CONNECTION POOLING

- Limits the number of open connections to a safe, efficient size
- **Key settings:** `maximumPoolSize`, `minimumIdle`, `idleTimeout`, `connectionTimeout`, `maxLifetime`



THREAD MANAGEMENT

- Too few threads means low throughput; too many means context switching and CPU pressure
- Use bounded thread pools and tune based on the actual workload



KEY AREAS THAT IMPACT PERFORMANCE

- Memory management and garbage collection tuning prevent `OutOfMemoryError` and reduce pauses
- JIT compilation, thread management, and blocking I/O all shape throughput and latency



GC OPTIONS

- **G1 (default):** balanced throughput and pause times, good for large heaps
- **ZGC / Shenandoah:** very low pause times for latency-sensitive services



HORIZONTAL SCALING (SCALE OUT)

- Add more instances behind a load balancer; distributes load across many nodes
- High availability and fault tolerance, but requires stateless applications



VERTICAL SCALING (SCALE UP)

- Add more CPU, RAM, and storage to a single node; simple, no code changes
- Hits a hardware ceiling and can require downtime to apply

- KEY TAKEAWAY** Creating a new database connection is expensive -- a connection pool reuses existing ones and prevents connection storms. •
- G1 is the sensible default for most microservices; ZGC and Shenandoah trade some throughput for very low pause times when latency is critical.
 - Horizontal scaling adds more instances behind a load balancer; vertical scaling adds more CPU, RAM, and storage to a single node.

BACKPRESSURE AND FLOW CONTROL · LOAD TESTING MICROSERVICES



HOW BACKPRESSURE WORKS

- Signals flow upstream to slow down the producer when the consumer or queue is overwhelmed
- Prevents buffer overflow and out-of-memory errors, keeping the system stable under load



KEY METRICS TO MONITOR

- Response time (avg, p95, p99), throughput, and error rate
- CPU/memory utilization, queue depth/lag, active connections and threads



FLOW CONTROL TECHNIQUES

- Rate limiting, bounded queues, credit-based flow control, reactive streams



COMMON TOOLS

- k6, JMeter, Gatling, Locust, Artillery, NBomber

KEY TAKEAWAY Without backpressure, unbounded queues grow until they exhaust memory; with it, the whole pipeline degrades gracefully instead of crashing. • Define objectives, plan, design, execute, analyze, then optimize and retest -- a six-step process turns load testing into a repeatable discipline.

STRESS, SPIKE, SOAK, AND SCALABILITY TESTING · SERVICE-LEVEL INDICATORS · SERVICE-LEVEL OBJECTIVES



THE FOUR TEST TYPES

- **Stress:** push beyond capacity until failure occurs, to find the breaking point
- **Spike:** sudden, extreme load increases, to validate graceful handling
- **Soak (endurance):** sustained load for 24-72+ hours, to catch memory/resource leaks
- **Scalability:** increase resources and measure throughput improvement



KEY CHARACTERISTICS OF GOOD SLOs

- User-focused, measurable, time-bound, challenging but achievable
- Limited in number -- a few critical objectives per service, not dozens



COMMON SLIS FOR MICROSERVICES

SLI	MEASURED AS
Latency	p50, p95, p99 (ms)
Availability	% of successful requests
Error rate	% of requests resulting in errors
Saturation	CPU %, memory %, queue size
Traffic/Throughput	Requests per second (RPS)



BANKING EXAMPLE

- **Payment Service latency (p95):** under 300 ms over a 7-day window
- **Order Service availability:** 99.9% over a 30-day window

KEY TAKEAWAY Stress finds the breaking point, spike checks sudden surges, soak reveals slow leaks, and scalability testing measures how well growth is absorbed. • An SLI is a quantitative measurement from the user's point of view -- the raw data every SLO and error budget is built on top of. • 99.9% availability over 30 days means: we expect the service to be available 99.9% of the time each month -- a clear, actionable commitment.

SERVICE-LEVEL AGREEMENTS · ERROR BUDGETS · CAPACITY PLANNING



KEY COMPONENTS OF AN SLA

- Service scope, service-level objectives, measurement methods
- Responsibilities of provider and consumer, and remediation actions if targets are missed



EXAMPLE SLA (ORDER SERVICE)

- 99.9% monthly uptime, P95 response time under 300 ms, error rate under 0.1%
- Incident response within 15 minutes, resolution within 4 hours



HOW IT WORKS

1. Define an SLO, e.g. 99.9% availability over a month
2. **Calculate the error budget:** 0.1% of the time is allowed for errors
3. Monitor burn rate in real time
4. Use the budget to guide release velocity and reliability investment



CONSEQUENCES OF BUDGET STATUS

- **Healthy:** safe to release new features and take more risk
- **Exhausted:** freeze new features, focus on reliability and remediation



KEY CAPACITY DRIVERS

- User demand, business growth, seasonality, data growth
- Performance goals and resilience needs (HA, failover, disaster recovery)



CAPACITY HEADROOM GUIDELINES

- **Low risk (stable demand):** 10-20% headroom
- **Mission critical:** 50-100%+ headroom

KEY TAKEAWAY Service scope, SLOs, measurement methods, responsibilities, and remediation together turn a reliability goal into an enforceable commitment.

- A 99.9% availability SLO over 30 days allows 43.2 minutes of downtime -- spend it wisely, and use burn rate to guide release risk.
- Understand demand, collect metrics, forecast, decide, and implement -- a five-step process keeps infrastructure sized to real, not guessed, need.

PERFORMANCE TESTING IN CI-CD

A quality gate with pass/fail thresholds on response time, throughput, and error rate turns performance into a release blocker, not an afterthought.



PERFORMANCE TESTING IN THE CI/CD PIPELINE

- Code commit -> build -> static analysis -> integration tests -> performance tests -> quality gate
- Quality gate evaluates metrics against thresholds before promoting to staging or production



BEST PRACTICES

- Automate performance tests in every pipeline run; fail the build when thresholds are breached
- Track performance trends over time, not just pass/fail on a single run

KEY TAKEAWAY A quality gate with pass/fail thresholds on response time, throughput, and error rate turns performance into a release blocker, not an afterthought.

EXERCISE 3.1 — CONFIGURE TIMEOUT AND CIRCUIT BREAKER

Module 6 checkpoint A — Resilience4j on Account dependency.



OBJECTIVE

- Resilience4j on Account dependency
- 10 min · code warmup checkpoint
- **Worksheet:**
curriculum/day-03/exercises/exercise-3.1-timeout-circuit-breaker.md



DO THIS

1. Find the RestClient timeouts (connect / read) in the Lab 3 Transaction starter
2. Circuit breaker opens after repeated Account failures; later calls fail fast
3. **Safe fallback:** POST /api/v1/transactions returns 503 ACCOUNT_SERVICE_UNAVAILABLE — no Kafka event
4. Bulkhead and rate limiting stay conceptual — do not install extra libraries for them



EXPECTED RESULT

- **You can state the fallback rule:** never invent a successful financial outcome.
- Lab 3 wraps AccountClient.requireActiveAccount. Fallback is 503, never a fake ACTIVE account.

KEY TAKEAWAY You can state the fallback rule: never invent a successful financial outcome.